LAPORAN PRAKTIKUM

PENGOLAHAN CITRA

(Dosen : *Rio Priantama S.T., M.T.I.*)

Modul 3

Nama : Muhammad Rizal Nurfirdaus

NIM : 20230810088

Kelas : TINFC-2023-04

TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS KUNINGAN

Praktikum

1. Praktikum 1
   Source Code:

```
unit Unit2;

interface

uses
  Winapi.Windows, Winapi.Messages, System.SysUtils, System.Variants,
  System.Classes, Vcl.Graphics, Vcl.Controls, Vcl.Forms, Vcl.Dialogs,
  Vcl.StdCtrls, Vcl.ExtCtrls, Vcl.ExtDlgs, Vcl.Imaging.jpeg,
  Vcl.Imaging.pngimage;

type
  TForm2 = class(TForm)
    ButtonMuatCitra: TButton;
    ButtonUbahWarna: TButton;
    LabelPosisi: TLabel;
    LabelMerah: TLabel;
    LabelHijau: TLabel;
    LabelBiru: TLabel;
    EditPosisi: TEdit;
    EditR: TEdit;
    EditG: TEdit;
    EditB: TEdit;
    Image1: TImage;
    OpenPictureDialog1: TOpenPictureDialog;
    procedure ButtonMuatCitraClick(Sender: TObject);
    procedure ButtonUbahWarnaClick(Sender: TObject);
    procedure Image1MouseDown(Sender: TObject; Button: TMouseButton;
      Shift: TShiftState; X, Y: Integer);
  private
    function TryMapImagePointToBitmap(ViewX, ViewY: Integer; out BmpX,
      BmpY: Integer): Boolean;
    function TryParsePositionText(const S: string; out X, Y: Integer): Boolean;
  end;

var
  Form2: TForm2;

implementation

{$R *.dfm}

procedure TForm2.ButtonMuatCitraClick(Sender: TObject);
var
```

```
  SrcPicture: TPicture;
  Bmp: TBitmap;
begin
  if OpenPictureDialog1.Execute then
  begin
    SrcPicture := TPicture.Create;
    Bmp := TBitmap.Create;
    try
      SrcPicture.LoadFromFile(OpenPictureDialog1.FileName);
      Bmp.Assign(SrcPicture.Graphic);
      Bmp.PixelFormat := pf24bit;
      Image1.Picture.Bitmap.Assign(Bmp);
      Image1.Repaint;
      EditPosisi.Clear;
      EditR.Clear;
      EditG.Clear;
      EditB.Clear;
    except
      on E: Exception do
        ShowMessage('Gagal memuat citra: ' + E.Message);
    end;
    Bmp.Free;
    SrcPicture.Free;
  end;
end;

function TForm2.TryMapImagePointToBitmap(ViewX, ViewY: Integer; out BmpX,
  BmpY: Integer): Boolean;
var
  BmpW, BmpH: Integer;
  DrawW, DrawH: Integer;
  OffsetX, OffsetY: Integer;
begin
  Result := False;
  BmpX := -1;
  BmpY := -1;

  if Image1.Picture.Bitmap.Empty then
    Exit;

  BmpW := Image1.Picture.Bitmap.Width;
  BmpH := Image1.Picture.Bitmap.Height;

  if (BmpW <= 0) or (BmpH <= 0) then
    Exit;
```

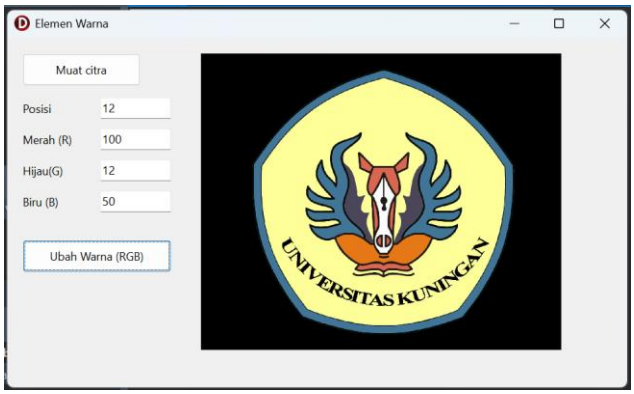
```
if Image1.Stretch then
begin
  if Image1.Proportional then
  begin
    if BmpW * Image1.Height > BmpH * Image1.Width then
    begin
      DrawW := Image1.Width;
      DrawH := Round(BmpH * Image1.Width / BmpW);
    end
    else
    begin
      DrawH := Image1.Height;
      DrawW := Round(BmpW * Image1.Height / BmpH);
    end;
    OffsetX := (Image1.Width - DrawW) div 2;
    OffsetY := (Image1.Height - DrawH) div 2;
  end
  else
  begin
    DrawW := Image1.Width;
    DrawH := Image1.Height;
    OffsetX := 0;
    OffsetY := 0;
  end;
end
else
begin
  DrawW := BmpW;
  DrawH := BmpH;
  OffsetX := 0;
  OffsetY := 0;
end;

if (DrawW <= 0) or (DrawH <= 0) then
  Exit;

if (ViewX < OffsetX) or (ViewY < OffsetY) or
  (ViewX >= OffsetX + DrawW) or (ViewY >= OffsetY + DrawH) then
  Exit;

BmpX := Trunc((ViewX - OffsetX) * BmpW / DrawW);
BmpY := Trunc((ViewY - OffsetY) * BmpH / DrawH);

if BmpX >= BmpW then
  BmpX := BmpW - 1;
if BmpY >= BmpH then
```

```
    BmpY := BmpH - 1;

  Result := (BmpX >= 0) and (BmpY >= 0);
end;

function TForm2.TryParsePositionText(const S: string; out X, Y: Integer): Boolean;
var
  P: Integer;
  SX, SY: string;
begin
  Result := False;
  X := -1;
  Y := -1;

  P := Pos(',', S);
  if P <= 1 then
    Exit;

  SX := Trim(Copy(S, 1, P - 1));
  SY := Trim(Copy(S, P + 1, MaxInt));

  if (SX = '') or (SY = '') then
    Exit;

  Result := TryStrToInt(SX, X) and TryStrToInt(SY, Y);
end;

procedure TForm2.ButtonUbahWarnaClick(Sender: TObject);
var
  X, Y: Integer;
  R, G, B: Integer;
  PC: PByteArray;
begin
  if Image1.Picture.Bitmap.Empty then
  begin
    ShowMessage('Muat citra terlebih dahulu.');
    Exit;
  end;

  if not TryParsePositionText(EditPosisi.Text, X, Y) then
  begin
    ShowMessage('Format posisi harus x,y (contoh: 10,15).');
    Exit;
  end;

  if not TryStrToInt(EditR.Text, R) then
```

```
  begin
    ShowMessage('Nilai R harus berupa angka 0-255.');
    Exit;
  end;

  if not TryStrToInt(EditG.Text, G) then
  begin
    ShowMessage('Nilai G harus berupa angka 0-255.');
    Exit;
  end;

  if not TryStrToInt(EditB.Text, B) then
  begin
    ShowMessage('Nilai B harus berupa angka 0-255.');
    Exit;
  end;

  if (X < 0) or (Y < 0) or
    (X >= Image1.Picture.Bitmap.Width) or (Y >= Image1.Picture.Bitmap.Height) then
  begin
    ShowMessage('Posisi berada di luar ukuran gambar.');
    Exit;
  end;

  if (R < 0) or (R > 255) or (G < 0) or (G > 255) or (B < 0) or (B > 255) then
  begin
    ShowMessage('Nilai RGB harus berada pada rentang 0 sampai 255.');
    Exit;
  end;

  PC := Image1.Picture.Bitmap.ScanLine[Y];
  PC[3 * X + 2] := Byte(R);
  PC[3 * X + 1] := Byte(G);
  PC[3 * X] := Byte(B);

  EditPosisi.Text := IntToStr(X) + ',' + IntToStr(Y);
  Image1.Repaint;
end;

procedure TForm2.Image1MouseDown(Sender: TObject; Button: TMouseButton;
  Shift: TShiftState; X, Y: Integer);
var
  BmpX, BmpY: Integer;
  PC: PByteArray;
  R, G, B: byte;
begin
```

```
  if (Image1.Picture.Bitmap.PixelFormat = pf24bit) and
    TryMapImagePointToBitmap(X, Y, BmpX, BmpY) then
  begin
    PC := Image1.Picture.Bitmap.ScanLine[BmpY];
    R := PC[3 * BmpX + 2];
    G := PC[3 * BmpX + 1];
    B := PC[3 * BmpX];
    EditPosisi.Text := IntToStr(BmpX) + ',' + IntToStr(BmpY);
    EditR.Text := IntToStr(R);
    EditG.Text := IntToStr(G);
    EditB.Text := IntToStr(B);
  end;
end;

end.
```

Hasil Run:



Analisis: Aplikasi pada gambar merupakan program pengolahan citra sederhana berbasis Delphi yang memungkinkan pengguna memuat gambar, melihat nilai warna piksel, serta mengubah warna pada koordinat tertentu. Tampilan antarmuka terdiri dari tombol untuk memuat citra, input posisi (x,y), nilai RGB, serta area **TImage** untuk menampilkan gambar (contohnya logo). Interaksi dibuat cukup intuitif: pengguna bisa langsung mengklik gambar untuk mendapatkan posisi dan nilai warna piksel, yang kemudian otomatis ditampilkan pada kolom input. Hal ini menunjukkan integrasi yang baik antara UI dan fungsi program.

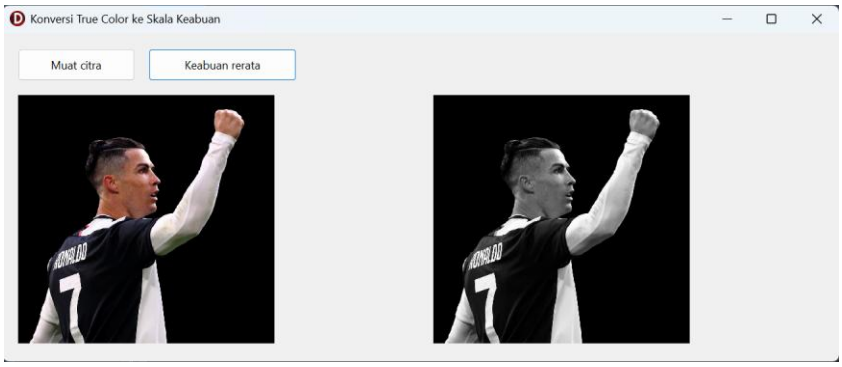
Dari sisi kode, program bekerja dengan mengonversi gambar ke format **bitmap pf24bit** agar bisa diakses per piksel menggunakan **ScanLine** dan **PByteArray**. Setiap piksel disimpan dalam format **BGR**, sehingga manipulasi warna dilakukan dengan indeks array tertentu. Fungsi tambahan seperti pemetaan koordinat memastikan klik pada gambar tetap akurat meskipun ukuran tampilan berbeda dari ukuran asli. Selain itu, validasi input (format posisi dan rentang RGB) membuat program lebih aman dari error. Secara keseluruhan, kode ini sudah menggambarkan konsep dasar pengolahan citra digital dengan manipulasi langsung pada data piksel.

2. Praktikum 2
   Source Code:
   unit Unit2;

```
interface

uses
  Winapi.Windows, Winapi.Messages, System.SysUtils, System.Variants,
  System.Classes, Vcl.Graphics, Vcl.Controls, Vcl.Forms, Vcl.Dialogs,
  Vcl.StdCtrls, Vcl.ExtCtrls, Vcl.ExtDlgs, Vcl.Imaging.jpeg,
  Vcl.Imaging.pngimage;

type
  TForm2 = class(TForm)
    ButtonMuatCitra: TButton;
    ButtonRerata: TButton;
    Image1: TImage;
    Image2: TImage;
    OpenPictureDialog1: TOpenPictureDialog;
    procedure ButtonMuatCitraClick(Sender: TObject);
    procedure ButtonRerataClick(Sender: TObject);
  end;

var
  Form2: TForm2;

implementation

{$R *.dfm}

procedure TForm2.ButtonMuatCitraClick(Sender: TObject);
var
  SrcPicture: TPicture;
  Bmp: TBitmap;
begin
  if OpenPictureDialog1.Execute then
  begin
    SrcPicture := TPicture.Create;
    Bmp := TBitmap.Create;
    try
      SrcPicture.LoadFromFile(OpenPictureDialog1.FileName);
      Bmp.Assign(SrcPicture.Graphic);
      Bmp.PixelFormat := pf24bit;
      Image1.Picture.Bitmap.Assign(Bmp);
      Image2.Picture.Bitmap.Assign(Bmp);
      Image2.Picture.Bitmap.PixelFormat := pf24bit;
      Image1.Repaint;
      Image2.Repaint;
    except
```

```
      on E: Exception do
        ShowMessage('Gagal memuat citra: ' + E.Message);
    end;
    Bmp.Free;
    SrcPicture.Free;
  end;
end;

procedure TForm2.ButtonRerataClick(Sender: TObject);
var
  PC, PH: PByteArray;
  X, Y: Integer;
  R, G, B, K: Byte;
begin
  if Image1.Picture.Bitmap.Empty then
  begin
    ShowMessage('Muat citra terlebih dahulu.');
    Exit;
  end;

  Image1.Picture.Bitmap.PixelFormat := pf24bit;
  Image2.Picture.Bitmap.Assign(Image1.Picture.Bitmap);
  Image2.Picture.Bitmap.PixelFormat := pf24bit;

  for Y := 0 to Image1.Picture.Bitmap.Height - 1 do
  begin
    PC := Image1.Picture.Bitmap.ScanLine[Y];
    PH := Image2.Picture.Bitmap.ScanLine[Y];
    for X := 0 to Image1.Picture.Bitmap.Width - 1 do
    begin
      R := PC[3 * X + 2];
      G := PC[3 * X + 1];
      B := PC[3 * X];
      K := Round((Integer(R) + Integer(G) + Integer(B)) / 3);
      PH[3 * X + 2] := K;
      PH[3 * X + 1] := K;
      PH[3 * X] := K;
    end;
  end;

  Image2.Repaint;
end;

end.
```

Hasil Run:



Analisis: Menurut saya, gambar dan aplikasi yang ditampilkan menunjukkan program konversi citra berwarna (true color) menjadi citra skala keabuan menggunakan metode rata-rata. Pada tampilan terlihat dua gambar: sebelah kiri adalah gambar asli (berwarna), sedangkan sebelah kanan adalah hasil konversi menjadi grayscale. Tombol **"Muat citra"** digunakan untuk memasukkan gambar, sementara tombol **"Keabuan rerata"** berfungsi untuk memproses dan mengubah citra menjadi abu-abu. Perubahan visual terlihat jelas, di mana warna pada gambar asli (misalnya warna kulit, baju, dan latar belakang) diubah menjadi variasi intensitas abu-abu berdasarkan tingkat kecerahannya.

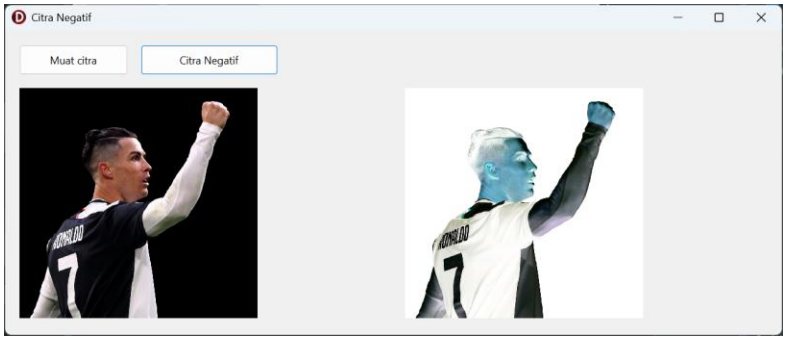
Dari sisi kode, proses konversi dilakukan dengan mengakses setiap piksel menggunakan **ScanLine** dan pointer **PByteArray** pada bitmap dengan format **pf24bit**. Nilai warna diambil dalam format **BGR**, kemudian dihitung nilai keabuan **K = (R + G + B) / 3**, yang merupakan metode rata-rata sederhana. Nilai K ini kemudian disalin ke semua kanal (R, G, B) pada gambar output (**Image2**), sehingga menghasilkan efek grayscale. Program ini sudah cukup baik karena memproses seluruh piksel menggunakan perulangan dua dimensi (X dan Y), serta memisahkan gambar input dan output, sehingga hasil konversi dapat langsung dibandingkan.

3. Praktikum 3
   Source Code:

```
unit Unit2;

interface

uses
  Winapi.Windows, Winapi.Messages, System.SysUtils, System.Variants,
  System.Classes, Vcl.Graphics, Vcl.Controls, Vcl.Forms, Vcl.Dialogs,
  Vcl.StdCtrls, Vcl.ExtCtrls, Vcl.ExtDlgs, Vcl.Imaging.jpeg,
  Vcl.Imaging.pngimage;

type
  TForm2 = class(TForm)
    ButtonMuatCitra: TButton;
    ButtonNegatif: TButton;
    Image1: TImage;
    Image2: TImage;
```

```
    OpenPictureDialog1: TOpenPictureDialog;
    procedure ButtonMuatCitraClick(Sender: TObject);
    procedure ButtonNegatifClick(Sender: TObject);
  end;

var
  Form2: TForm2;

implementation

{$R *.dfm}

procedure TForm2.ButtonMuatCitraClick(Sender: TObject);
var
  SrcPicture: TPicture;
  Bmp: TBitmap;
begin
  if OpenPictureDialog1.Execute then
  begin
    SrcPicture := TPicture.Create;
    Bmp := TBitmap.Create;
    try
      SrcPicture.LoadFromFile(OpenPictureDialog1.FileName);
      Bmp.Assign(SrcPicture.Graphic);
      Bmp.PixelFormat := pf24bit;
      Image1.Picture.Bitmap.Assign(Bmp);
      Image2.Picture.Bitmap.Assign(Bmp);
      Image2.Picture.Bitmap.PixelFormat := pf24bit;
      Image1.Repaint;
      Image2.Repaint;
    except
      on E: Exception do
        ShowMessage('Gagal memuat citra: ' + E.Message);
    end;
    Bmp.Free;
    SrcPicture.Free;
  end;
end;

procedure TForm2.ButtonNegatifClick(Sender: TObject);
var
  PC, PH: PByteArray;
  X, Y: Integer;
  R, G, B: Byte;
begin
  if Image1.Picture.Bitmap.Empty then
```

```
  begin
    ShowMessage('Muat citra terlebih dahulu.');
    Exit;
  end;

  Image1.Picture.Bitmap.PixelFormat := pf24bit;
  Image2.Picture.Bitmap.Assign(Image1.Picture.Bitmap);
  Image2.Picture.Bitmap.PixelFormat := pf24bit;

  for Y := 0 to Image1.Picture.Bitmap.Height - 1 do
  begin
    PC := Image1.Picture.Bitmap.ScanLine[Y];
    PH := Image2.Picture.Bitmap.ScanLine[Y];
    for X := 0 to Image1.Picture.Bitmap.Width - 1 do
    begin
      R := PC[3 * X + 2];
      G := PC[3 * X + 1];
      B := PC[3 * X];
      PH[3 * X + 2] := 255 - R;
      PH[3 * X + 1] := 255 - G;
      PH[3 * X] := 255 - B;
    end;
  end;

  Image2.Repaint;
end;

end.
```



Analisis : Saat saya membuat program ini, saya ingin membuat aplikasi sederhana untuk memuat gambar dan mengubahnya menjadi citra negatif. Saya menggunakan dua komponen gambar, yaitu Image1 untuk menampilkan citra asli dan Image2 untuk hasilnya. Ketika saya menekan tombol *Muat Citra*, saya memilih gambar dari komputer, lalu mengubahnya ke format bitmap 24-bit agar bisa diproses, dan menampilkannya di kedua area.

Ketika tombol *Citra Negatif* ditekan, saya memproses setiap piksel menggunakan metode ScanLine agar lebih cepat. Saya mengambil nilai warna RGB (dalam format BGR) lalu mengubahnya dengan rumus 255 dikurangi nilai aslinya untuk menghasilkan

efek negatif. Hasilnya langsung ditampilkan di Image2, sementara gambar asli tetap aman di Image1.

LATIHAN/TUGAS

1. Jelaskan kesimpulan dari praktikum modul 3 ini!
   Menurut saya, kesimpulan dari praktikum modul 3 ini adalah bahwa pengolahan citra digital pada dasarnya bekerja dengan memanipulasi nilai setiap piksel secara langsung. Melalui penggunaan **TBitmap dengan format pf24bit**, kita dapat mengakses data citra secara low-level menggunakan **ScanLine** dan **PByteArray**, di mana setiap piksel direpresentasikan dalam format **BGR (Blue, Green, Red)**. Dari praktikum yang telah dilakukan—mulai dari membaca nilai warna piksel melalui klik pada gambar, mengubah warna piksel tertentu berdasarkan koordinat, mengonversi citra menjadi grayscale dengan metode rata-rata, hingga membuat efek negatif—terlihat bahwa semua proses tersebut hanya melibatkan operasi sederhana terhadap nilai RGB, tetapi menghasilkan perubahan visual yang signifikan pada gambar.
   Selain itu, praktikum ini juga menekankan pentingnya aspek teknis dalam pengembangan aplikasi pengolahan citra, seperti validasi input (format koordinat dan rentang nilai RGB), pemetaan koordinat antara tampilan dan bitmap asli (terutama saat gambar di-stretch atau diperkecil), serta efisiensi pengolahan dengan looping dua dimensi (X dan Y). Penggunaan dua komponen **TImage** (input dan output) juga menunjukkan konsep pemisahan data asli dan hasil proses agar lebih aman dan mudah dibandingkan. Secara keseluruhan, modul ini memberikan pemahaman menyeluruh tentang dasar-dasar **image processing**, baik dari sisi teori (konsep RGB, grayscale, negatif) maupun implementasi (manipulasi piksel langsung di Delphi), sehingga menjadi fondasi penting untuk pengembangan teknik lanjutan seperti filtering, segmentasi, dan computer vision.
2. Jelaskan cara membuat main menu pada program!
   Cara saya membuat **Main Menu** pada program Delphi ini cukup sederhana karena sudah disediakan komponen bawaan dari VCL. Pertama, saya menambahkan komponen **TMainMenu** dari *Tool Palette* ke dalam form. Setelah itu, saya mulai membuat struktur menu dengan mengklik komponen tersebut lalu menambahkan item seperti **File**, **Process**, dan **Help**. Di dalam menu *File*, saya tambahkan submenu seperti *Load Image*, *Save Result*, dan *Exit*. Kemudian pada menu *Process*, saya tambahkan *Negative* dan *Grayscale*, sedangkan pada *Help* saya tambahkan *About*. Struktur ini saya sesuaikan dengan fitur yang ingin saya buat di aplikasi.
   Selanjutnya, saya menghubungkan setiap item menu dengan kode program melalui event **OnClick**. Caranya, saya cukup double-click pada masing-masing menu item, lalu Delphi otomatis membuat prosedur seperti MenuLoadImageClick, MenuNegativeClick, dan lainnya. Di dalam prosedur tersebut, saya isi logika sesuai fungsi menu, misalnya untuk membuka gambar, memproses citra, atau menyimpan hasil. Dengan cara ini, setiap menu yang saya klik akan menjalankan perintah tertentu, sehingga aplikasi menjadi lebih terstruktur dan mudah digunakan.